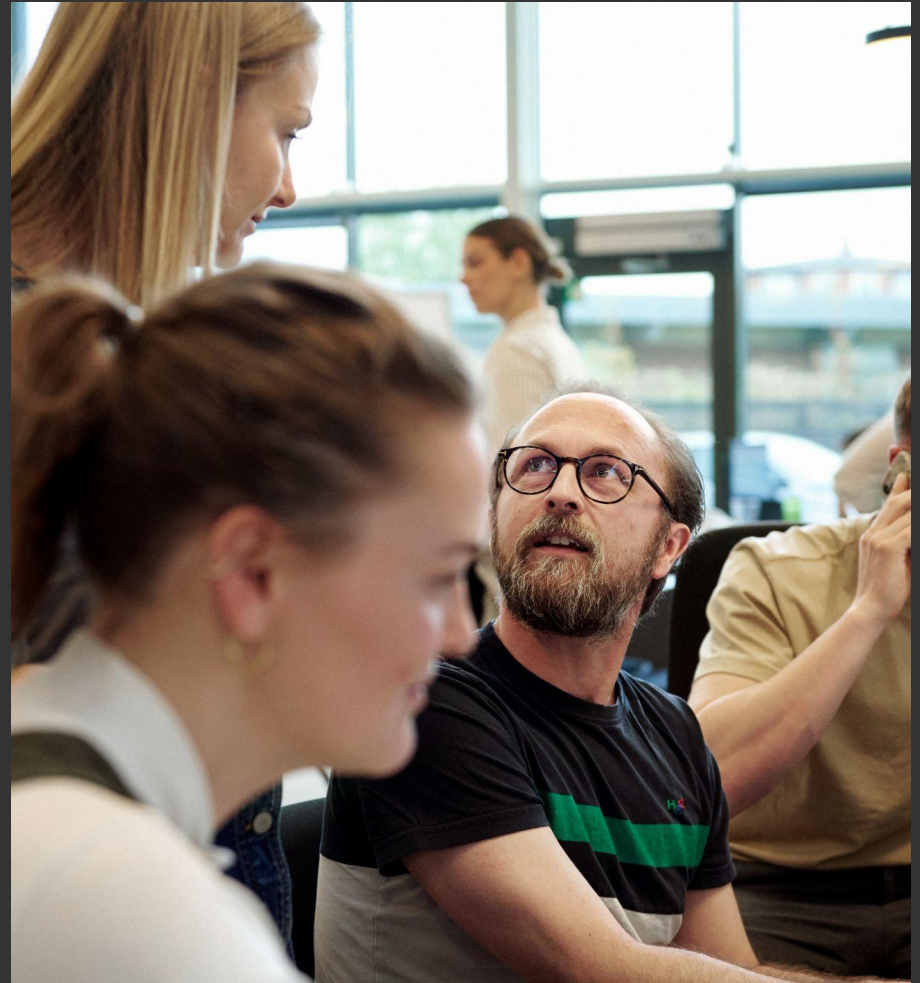


Python Libraries & Frameworks

Course 11

2025



agenda for today

libraries vs frameworks

examples

library

A library is set of related, low-level **components that developers invoke to achieve a specific outcome.**

Common library functions include:

- date formatting
- establishing a network connection
- file processing



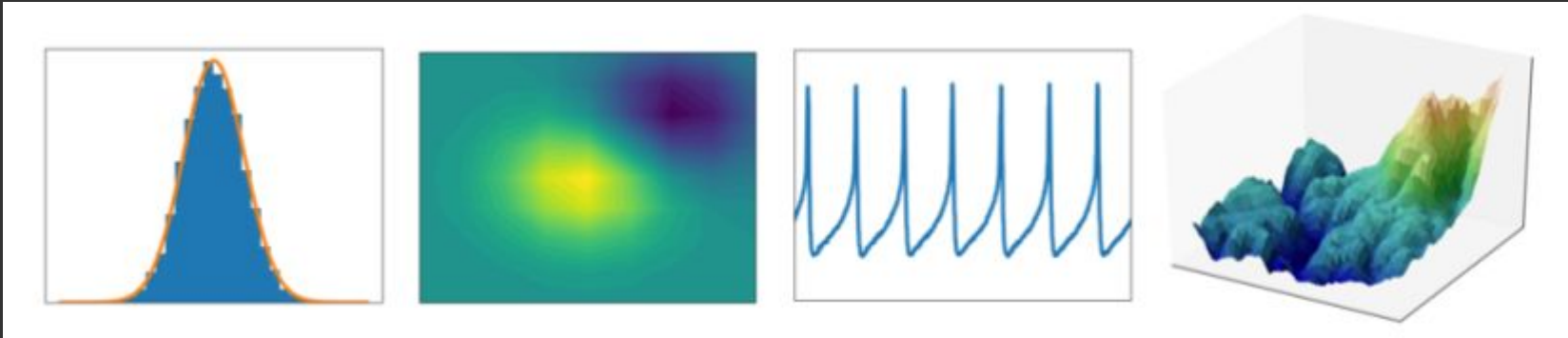
Libraries

1. **Import Libraries.** Begin by importing libraries using the import statement. You can import entire libraries or specific modules within a library.
2. **Read Documentation.** Familiarize yourself with the documentation of the libraries you use. Documentation provides details about available functionalities, parameters, return values, and usage examples.
3. **Utilize Functions and Classes.** Use imported functions, classes, and other objects provided by the library in your program as needed.
4. **Manage Dependencies.** Use tools like **pip** to install required libraries and their dependencies.



Libraries

matplotlib: A plotting library for creating static, animated, and interactive visualizations in Python. It provides an object-oriented API for embedding plots into applications.



Libraries

numpy: is the fundamental package for scientific computing with Python.

Provides routines for fast operations on arrays, including mathematical, logical, shape manipulation, sorting, selecting, I/O, discrete Fourier transforms, basic linear algebra, basic statistical operations, random simulation and much more.

```
import numpy as np

# Create a 2-D array, set every second element in
# some rows and find max per row:
x = np.arange(15, dtype=np.int64).reshape(3, 5)
x[1:, ::2] = -99

x
# array([[ 0,  1,  2,  3,  4],
#        [-99,  6, -99,  8, -99],
#        [-99, 11, -99, 13, -99]])

x.max(axis=1)
# array([ 4,  8, 13])

# Generate normally distributed random numbers:
rng = np.random.default_rng()
samples = rng.normal(size=2500)
```

Libraries

requests: This library makes dealing with HTTPS requests a breeze.

Requests provides functions for interacting with web APIs and managing HTTP responses.

```
import requests

url = "https://api.github.com/users/example"

# Making a GET request to the API
response = requests.get(url)

# Checking if the request was successful
if response.status_code == 200:
    data = response.json()

    username = data['login']
    user_id = data['id']
    profile_url = data['html_url']

    print(f"Username: {username}")
    print(f"User ID: {user_id}")
    print(f"Profile URL: {profile_url}")
else:
    print(f"Failed to retrieve data: Status code {response.status_code}")
```

Libraries

dateTime: is a standard Python library that is essential for dealing with any DateTime format.

```
import datetime

date1 = datetime.date(2024, 4, 22)
time1 = datetime.time(14, 30)
datetime1 = datetime.datetime(2024, 4, 22, 14, 30)

print(f"Date: {date1}")
print(f"Time: {time1}")
print(f"DateTime: {datetime1}")
```

Libraries

unittest: is Python's built-in testing framework. Its key features are: test discovery, support for fixtures, effortless organization, and management of test suites.

pytest: is a framework that simplifies test writing and execution. It is user-friendly with its concise syntax.

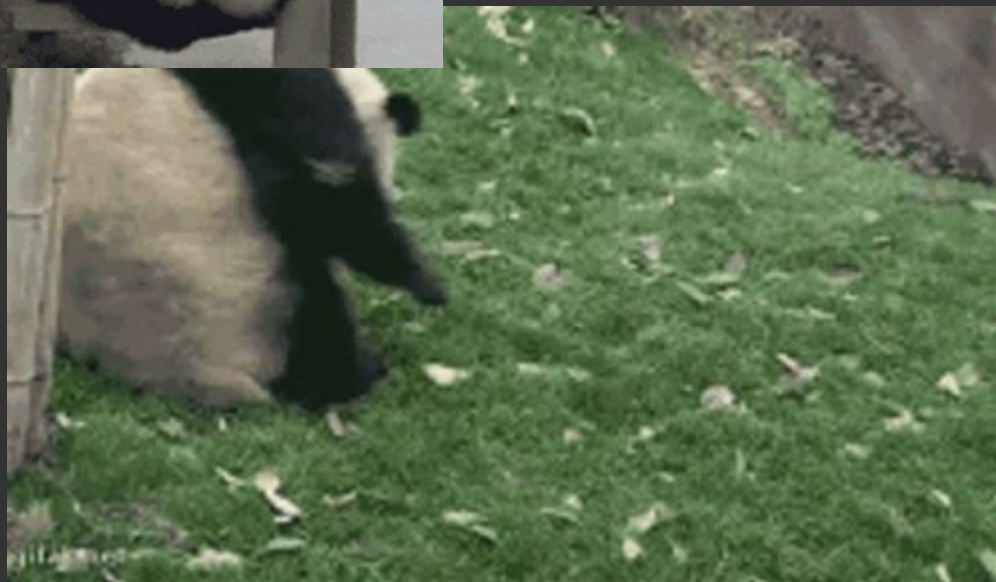
```
# file: math_functions.py
def add_numbers(x, y):
    return x + y

# file: test_math_functions.py
import unittest
from math_functions import add_numbers

class TestMathFunctions(unittest.TestCase):
    def test_add_positive_numbers(self):
        self.assertEqual(add_numbers(3, 4), 7)

    def test_add_zero_and_negative_number(self):
        self.assertEqual(add_numbers(-1, 1), 0)

    def test_add_negative_numbers(self):
        self.assertEqual(add_numbers(-2, -2), -4)
```



Libraries

pandas: Pandas is a Python library that helps you work with structured data effortlessly. It's like having a superpower for manipulating and analyzing data tables.





1. **Read and Write Data:** With Pandas, you can easily read data from different sources like CSV files, Excel spreadsheets, databases, and more. You can also write your processed data back to these formats.
2. **Explore Your Data:** Pandas allows you to quickly peek into your data, see what it looks like, and understand its structure. You can check the first few rows, the last few rows, and get a summary of your data.
3. **Clean Your Data:** Data is often messy! Pandas helps you clean up missing values, remove duplicates, and fix inconsistencies in your data so that you can trust your analysis.
4. **Manipulate Your Data:** Need to add a new column, rename columns, or filter rows based on certain conditions? Pandas makes it easy to manipulate your data in various ways without breaking a sweat.
5. **Analyze Your Data:** Once your data is ready, you can perform all sorts of analysis with Pandas. Calculate statistics, group data by categories, and even merge different datasets together.
6. **Visualize Your Data:** Want to see trends, patterns, or relationships in your data? Pandas works seamlessly with visualization libraries like Matplotlib and Seaborn to create beautiful plots and charts.



Efficiency: Pandas simplifies complex data tasks into just a few lines of code, saving you time and effort.

Versatility: Whether you're a data scientist, analyst, or just curious about data, Pandas is a valuable tool for working with any kind of structured data.

Integration: Pandas plays well with other Python libraries like NumPy, Matplotlib, and Scikit-learn, forming a powerful ecosystem for data science and analysis.



Let's use an example of a CSV file (cars.csv):

```
CarID,Make,Model,Year,Color,Mileage,Price,Engine Type,Transmission
1,Ford,Mustang,2018,Red,45000,25000,V6,Manual
2,Chevrolet,Camaro,2017,Black,37000,23000,V8,Automatic
3,Toyota,Corolla,2020,White,22000,18000,I4,Automatic
4,Honda,Civic,2019,Blue,31000,20000,I4,Manual
5,Tesla,Model 3,2021,Black,15000,35000,Electric,Automatic
6,Volvo,V60,2024,Gray,0,45000,T8 Hybrid,Automatic
7,Ford,F-150,2019,Blue,60000,27000,V8,Automatic
8,Chevrolet,Impala,2018,Silver,32000,22000,V6,Automatic
9,Toyota,Prius,2021,Green,10000,23000,I4,Automatic
10,Honda,Accord,2022,White,5000,28000,I4,Automatic
```



Example 1. Load cars from CSV file:

```
import pandas as pd

df_cars = pd.read_csv('cars.csv')
print(df_cars)
```

Output:

	CarID	Make	Model	Year	Color	Mileage	Price	Engine	Type	Transmission
0	1	Ford	Mustang	2018	Red	45000	25000		V6	Manual
1	2	Chevrolet	Camaro	2017	Black	37000	23000		V8	Automatic
2	3	Toyota	Corolla	2020	White	22000	18000		I4	Automatic
3	4	Honda	Civic	2019	Blue	31000	20000		I4	Manual
4	5	Tesla	Model 3	2021	Black	15000	35000		Electric	Automatic
5	6	Volvo	V60	2024	Gray	0	45000		T8 Hybrid	Automatic
6	7	Ford	Fiesta	2016	Green	60000	8000		I4	Manual
7	8	Toyota	Camry	2018	Silver	29000	23000		V6	Automatic
8	9	Chevrolet	Spark	2015	Yellow	50000	10000		I4	Manual
9	10	Ford	Explorer	2022	Blue	10000	30000		V6	Automatic



Example 2. Filter Cars by Color and Year

```
import pandas as pd

df_cars = pd.read_csv('cars.csv')
black_cars_2020_onwards = df_cars[(df_cars['Color'] == 'Black') & (df_cars['Year'] >= 2020)]
print(black_cars_2020_onwards)
```

Output:

	CarID	Make	Model	Year	Color	Mileage	Price	Engine Type	Transmission
4	5	Tesla	Model 3	2021	Black	15000	35000	Electric	Automatic



Example 3. Group by Make and Calculate Average Mileage and Price

```
import pandas as pd

df_cars = pd.read_csv('cars.csv')
avg_mileage_price = df_cars.groupby('Make').agg({
    'Mileage': 'mean',
    'Price': 'mean'
}).rename(columns={'Mileage': 'Average Mileage', 'Price': 'Average Price'})
print(avg_mileage_price)
```

Output:

	Average Mileage	Average Price
Make		
Chevrolet	43500.000000	16500.0
Ford	38333.333333	21000.0
Honda	31000.000000	20000.0
Tesla	15000.000000	35000.0
Toyota	25500.000000	20500.0
Volvo	0.000000	45000.0



Example 4. Count Cars by Color

```
import pandas as pd

df_cars = pd.read_csv('cars.csv')
color_counts = df_cars['Color'].value_counts()
print(color_counts)
```

Output:

Black	2
Blue	2
Red	1
White	1
Gray	1
Green	1
Silver	1
Yellow	1



Example 5. Find the Newest Car for Each Make

```
import pandas as pd

df_cars = pd.read_csv('cars.csv')
newest_cars = df_cars.groupby('Make').apply(lambda x: x[x['Year'] == x['Year'].max()])
print(newest_cars[['Make', 'Model', 'Year']])
```

Output:

	Make	Model	Year
Make			
Chevrolet	Chevrolet	Camaro	2017
Ford	Ford	Explorer	2022
Honda	Honda	Civic	2019
Tesla	Tesla	Model 3	2021
Toyota	Toyota	Corolla	2020
Volvo	Volvo	V60	2024



Example 6. Calculate Depreciation (rate - 10% yearly)

```
import pandas as pd

df_cars = pd.read_csv('cars.csv')
df_cars['Depreciation'] = df_cars.apply(lambda x: x['Price'] * 0.1 * (2024 - x['Year']), axis=1)
print(df_cars[['Make', 'Model', 'Year', 'Price', 'Depreciation']])
```

Output:

	Make	Model	Year	Price	Depreciation
0	Ford	Mustang	2018	25000	15000.0
1	Chevrolet	Camaro	2017	23000	16100.0
2	Toyota	Corolla	2020	18000	7200.0
3	Honda	Civic	2019	20000	10000.0
4	Tesla	Model 3	2021	35000	10500.0
5	Volvo	V60	2024	45000	0.0
6	Ford	Fiesta	2016	8000	6400.0
7	Toyota	Camry	2018	23000	13800.0
8	Chevrolet	Spark	2015	10000	9000.0
9	Ford	Explorer	2022	30000	6000.0



Example 7. Writing to file

```
import pandas as pd

df_cars = pd.read_csv('cars.csv')
df_cars['Depreciation'] = df_cars.apply(lambda x: x['Price'] * 0.1 * (2024 - x['Year']), axis=1)

df_cars.to_csv('updated_cars.csv', index=False)
```

Output:

```
Make,Model,Year,Color,Mileage,Price,Engine Type,Transmission,Depreciation
Ford,Mustang,2018,Red,45000,25000,V6,Manual,15000.0
Chevrolet,Camaro,2017,Black,37000,23000,V8,Automatic,16100.0
Toyota,Corolla,2020,White,22000,18000,I4,Automatic,7200.0
Honda,Civic,2019,Blue,31000,20000,I4,Manual,10000.0
Tesla,Model 3,2021,Black,15000,35000,Electric,Automatic,10500.0
Volvo,V60,2024,Gray,0,45000,T8 Hybrid,Automatic,0.0
Ford,Fiesta,2016,Green,60000,8000,I4,Manual,6400.0
Toyota,Camry,2018,Silver,29000,23000,V6,Automatic,13800.0
Chevrolet,Spark,2015,Yellow,50000,10000,I4,Manual,9000.0
Ford,Explorer,2022,Blue,10000,30000,V6,Automatic,6000.0
```

Frameworks

A framework provides a **foundational structure** for developing software.

Acts as a **skeleton where developers add their own code** to create fully functional software.

Enables rapid development with predefined structures and practices.



Frameworks

Unlike libraries that offer specific functionality, a framework **dictates the architecture** of your application.

Provides essential scaffolding which developers build upon.

Includes tools and components that standardize the development process.



Inversion of Control

Fundamental to many frameworks is the **Inversion of Control** (IoC) concept.

- In libraries, the code calls the library functions.
- In frameworks, **the framework calls your code.**

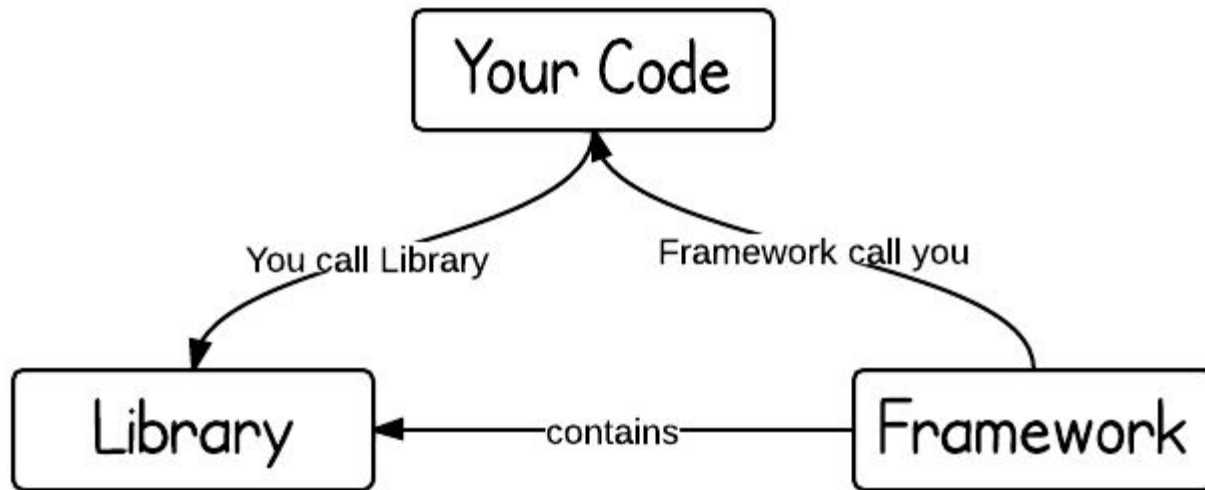


"don't call us;
we'll call you"

- Hollywood principle

a.k.a. **Inversion of Control** (IoC) principle





Library

The major difference between frameworks and libraries is **complexity**.

A library is a collection of packages that performs **specific operations**.

Framework

Frameworks contain the **basic flow** and **architecture** of the application

Library

Control is also a key difference between these two.

When a developer calls a method from a library, the developer is in control of the development process

Framework

With frameworks, it is the one calling.

It means that the frameworks call the processes, and the control lies in the framework.

Example:
Kitchen tools / gadgets



Example:
Recipe book



Library

Example:

Kitchen tools / gadgets

Imagine you're cooking a meal in your kitchen.

A library is like the array of kitchen tools at your disposal — spatulas, knives, measuring cups.

You choose which tool to use based on the specific task at hand, such as chopping vegetables or stirring a sauce.

The utensils don't tell you what to cook or how to cook it; they simply assist in the tasks you decide to undertake.

Framework

Example:

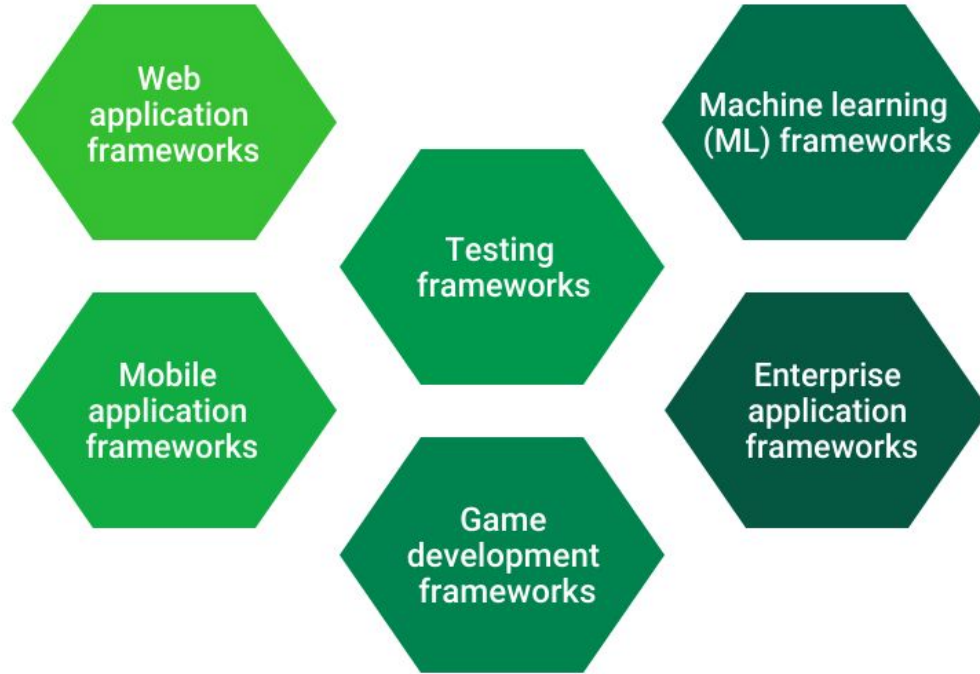
Recipe book

In contrast, a framework is like following a recipe from a cookbook.

The recipe provides a structured approach to cooking a dish, specifying the ingredients, the order of steps, and the cooking methods to use.

While you can make minor tweaks, the basic structure and flow of the cooking process are dictated by the recipe, guiding you to the expected outcome.

Frameworks



Components of Frameworks

Frameworks include a variety of components that are common across many applications.

Examples include:

- Database Interaction Tools: ORM (Object-Relational Mapping)
- Routing Mechanisms: URL routing for web applications
- Template Engines: for rendering views

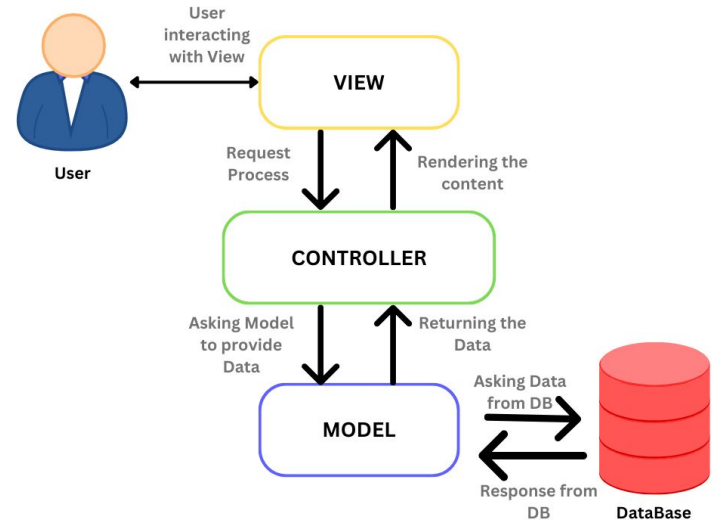
These components support a range of functionalities and ease the development process.

Frameworks architecture

Frameworks provide a pre-defined architecture for building applications.

Encourages developers to organize code in specific ways that align with the framework's design.

Typically involves a **Model-View-Controller (MVC)** pattern in web frameworks like Django.



django

Django is a high-level Python web framework that encourages **rapid development and clean, pragmatic design**.

Was designed to help developers take applications from concept to completion as quickly as possible.

Example 1. Setting Up a Django Project

```
pip install django
django-admin startproject myproject
cd myproject
python manage.py startapp myapp
```

django Key Features

Django follows a **"batteries-included"** approach, meaning it comes with a multitude of built-in features for common web development tasks.

This includes:

- user authentication,
- content administration,
- site maps,
- RSS feeds, and many more functionalities that are ready to use right out of the box.

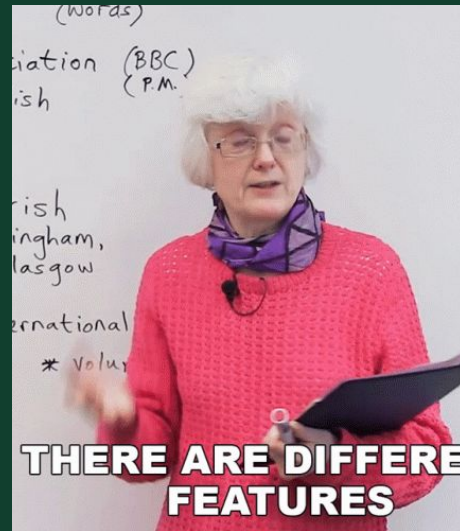


django Key Features

Automatic Admin Interface

One of Django's most celebrated features is its automatically-generated admin interface.

It's a dynamic, production-ready admin interface that can be set up with minimal coding, allowing developers to manage website content quickly and efficiently.



djan

Automat

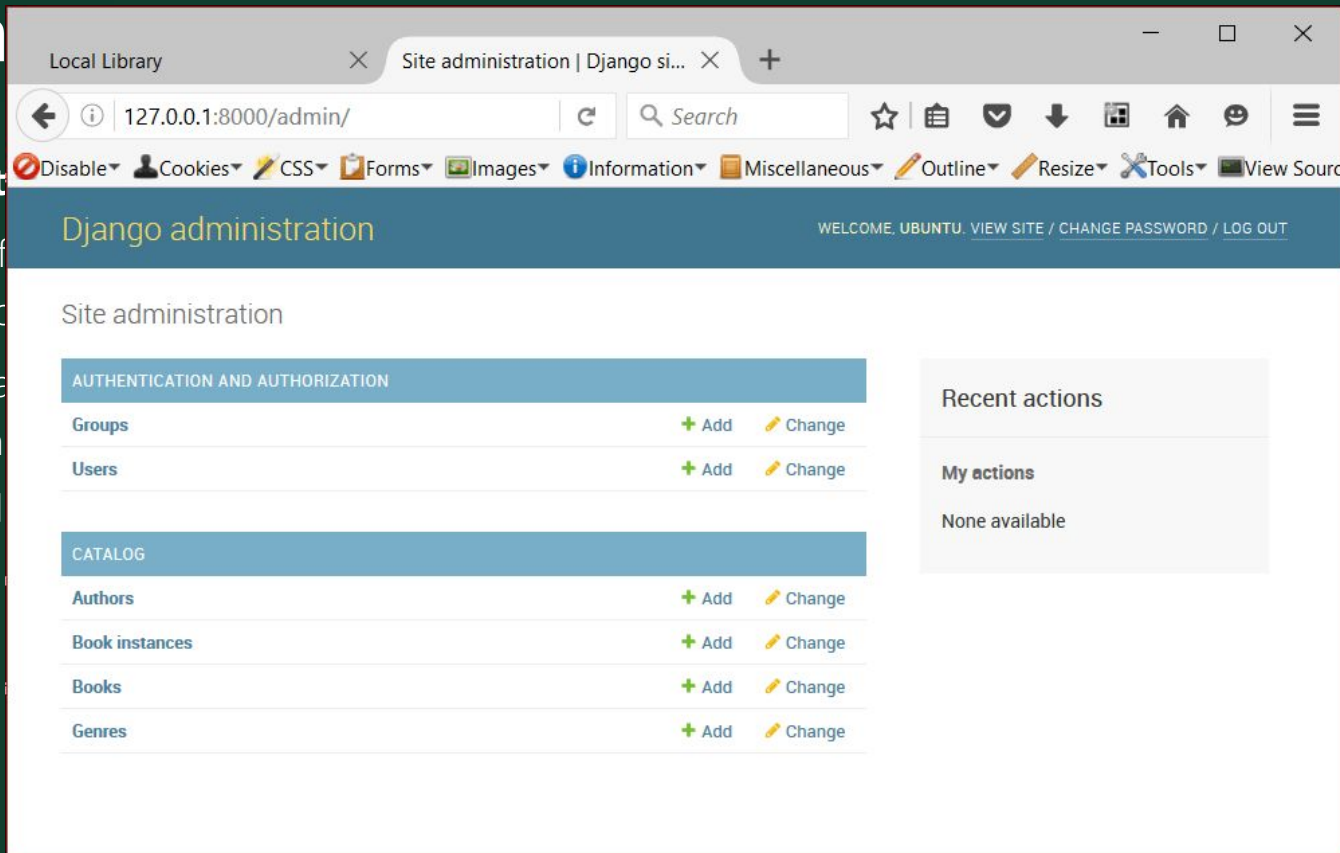
One of

automatic

It's a dyna

with min

content q



THERE ARE DIFFERENT FEATURES

django Key Features

Security Features

Django emphasizes security and helps developers avoid many common security mistakes, such as SQL injection, cross-site scripting, cross-site request forgery, and clickjacking.

Its user authentication system provides a secure way to manage user accounts and passwords.



django Key Features

Vibrant Community & Extensive Documentation

Django has some of the best documentation of any open-source framework. It is comprehensive, updated regularly, and provides a great resource for both novice and experienced developers alike.

A large community of developers supports Django. This community contributes to a rich ecosystem of packages and plugins, making it possible to customize and extend your Django application to perform a wide variety of tasks.





Example 2. Configuring URLs

In Django, URLs are configured in `urls.py`. Here's how you can link the above view to a URL:

```
from django.urls import path
from .views import book_list

urlpatterns = [
    path('books/', book_list, name='book-list'),
]
```



Example 3. Creating Views

Views handle the business logic of your application. Below is a simple view that returns a list of all books:

```
from django.http import HttpResponse
from .models import Book

def book_list(request):
    books = Book.objects.all()
    book_titles = ', '.join([book.title for book in books])
    return HttpResponse(f"Books available: {book_titles}")
```



Example 4. Template Usage

Django uses the MVT (Model-View-Template) architecture. Here's a simple template example that could be saved as `books.html` in the `templates` directory:

```
<!DOCTYPE html>
<html>
<head>
  <title>Book List</title>
</head>
<body>
  <h1>Books</h1>
  <ul>
    {% for book in books %}
      <li>{{ book.title }} by {{ book.author }}</li>
    {% endfor %}
  </ul>
</body>
</html>
```

```
from django.shortcuts import render
from .models import Book

def book_list(request):
    books = Book.objects.all()
    return render(request, 'books.html', {'books': books})
```

- Python is too slow.
- Python does not scale for millions of API requests
- It is not concurrent / multi threaded
- No company uses Python for scale.
You can only use JAVA or Golang.







Instagram Scales on Python for 2 Billion Daily Users



Shrey Batra

Founder & CEO @ Cosmocloud | Ex-LinkedIn | Patent for Distributed Algorithm | MongoDB Champion

68 articles

✓ Following

March 30, 2022

Instagram & OOP & python: **success** story

Instagram, one of the most famous Social Media apps of our time, sharing enormous amount of photos, videos and media content, STILL RUNS ON PYTHON..!

One of our favourite social media platform still works on a Monolithic Python's Django codebase..



Instagram's Scale

Instagram had a scale of **95 Million** photos and videos uploaded every day, **back in 2016**.

Now, with that scale, back in 2016, they also had about **4.2 Billion Likes** being sent **every single day..!**

Now, to handle this scale for every single day, Instagram has a scale of 10's of thousands of **Django** Servers running, scattered across the world, fault tolerant and so much more..!



Instagram's History with Django

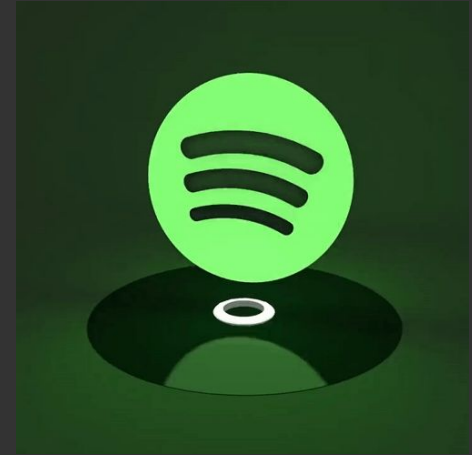
Instagram started with Django back in 2010 as their go to tech stack.

Why? Because it's **super easy to setup and took only 2 weeks to start** and a couple of months to reach a million users..!

Only in 2011 Summer, Instagram had about 5M users with all of its data being stored in PostgreSQL and using Django's ORM.



Have you ever heard to music being played on Spotify? Do you know what powers Spotify's backend APIs?



Youtube is another classic example of a company who initially were using PHP but soon migrated to Django and Python.

Google has used Python since its early days, leveraging its simplicity and versatility for a wide range of applications, including web search algorithms and system administration tasks.

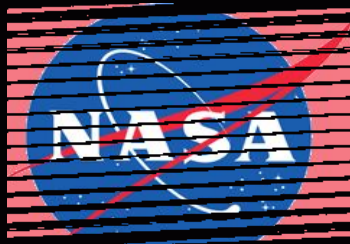
While specific figures related to Python's impact at Google are not publicly detailed, Google's search engine processes over **3.5 billion searches per day**, a testament to the scalability and efficiency of its underlying technologies, including Python.



Netflix uses Python for server-side data analysis. It is pivotal in data processing pipelines, security automation, and risk classification.

Netflix streams billions of hours of content monthly to over 200 million subscribers worldwide, with **Python enabling sophisticated data analysis and operational automation.**

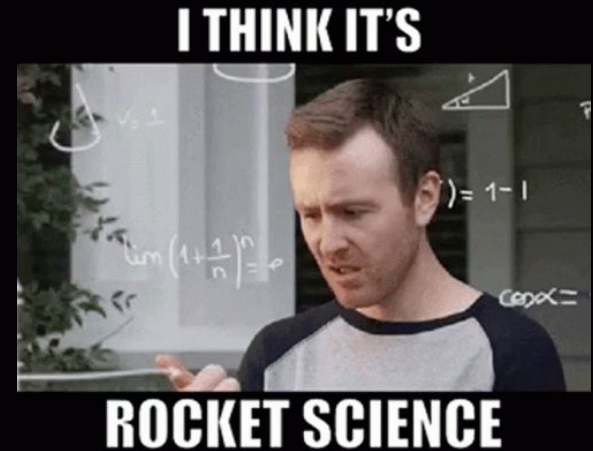
NETFLIX



NASA uses Python for a variety of tasks, including data analysis, scientific computing, and system integration.

Python's libraries and frameworks support NASA's need for **complex data processing, simulation, and computational modeling.**

Python has been used in numerous space missions and projects, including the Mars Rover's navigation system and the ARES Division (Atmospheric Reentry Engineering Simulation), demonstrating its reliability in critical and high-precision environments.





Flask - alternative for Django

web development,
one drop at a time

Flask is a popular, lightweight Python web framework known for its simplicity and flexibility.

It is classified as a **microframework** because it does not require particular tools or libraries.

Comes with a minimal set of components and extensions, making it easy for newcomers to get started quickly without having to make decisions about which additional components to use.

Brands using **Flask**



**NO MORE
CONTENT**





Entrepreneurial Responsible Dedicated Inclusive



Champions of cloud software

